LibSpace

Project Title:

- Libspace API - A Robust Backend for Library Operations.

Project id:SWTID-2026-3352

Term Leader:

- N.kamali Email id: ndkamali12@gmail.com

Term Members:

- A.Hemavathi Email id: hemavathia475@gmail.com
- S.Sirajunnisha Email id: sirajunnisha2007@gmail.com
- V.kalaiarasi Email id: kalaiarasi251206@gmail.com

1. Project Overview

LibSpace is a micro web-based library management system that provides secure user authentication and basic book management features. The application allows users to register, log in, log out, and delete their accounts. Once authenticated, users can perform CRUD (Create, Read, Update, Delete) operations on books, including adding new books, viewing the list of books, updating book details, and deleting books.

The main goal of LibSpace is to demonstrate authentication flow and CRUD operations while backend, and database functionalities in a structured manner.

2. Project Instructions – *LibSpace*

1. Register a new user account using the registration form.
2. Log in with the registered credentials to access the dashboard.
3. After successful login, perform book management operations:
    - Add a new book.
    - View the list of available books.
    - Get book by ID
    - Update book details.
    - Delete a book.

4. Users can log out securely after completing operations.
5. Users also have the option to delete their account permanently from the system.

These instructions describe how the system should be used once it is running.

❖ Technical Architecture

LibSpace follows a **Layered Backend Architecture** consisting of:

1. **API Layer (Routes)**
2. **Business Logic Layer (Controllers)**
3. **Data Access Layer (Models / Database)**

This structure ensures clean code organization, scalability, and maintainability.

❖ Features

1. User Authentication

- User Registration (name, email, password)
- Secure password storage (hashed password)
- User Login with credential verification
  - Token-based authentication (JWT)
  - Delete User Account
    - Automatic timestamps (createdAt, updatedAt)

2. Book Management (CRUD Operations)

Each book is linked to a specific user using a reference (`user: ObjectId`).

- Add a new book (url, title, author)
- Get all books of a specific user
- Get book by ID
- Update book details
- Delete a book
- Books are protected and accessible only to the authenticated user

3. User–Book Relationship

- One-to-Many Relationship:
  - One User → Can have multiple Books
  - Each Book → Belongs to one User (via ObjectId reference)
- Maintains data integrity using MongoDB references

4. Security & Validation

- Input validation for required fields
- Protected routes using authentication middleware
- Error handling for invalid requests
- Unauthorized access prevention

5. RESTful API Design

- Follows REST principles
- Uses standard HTTP methods:
  - POST
  - GET
  - PUT
  - DELETE
- Returns structured JSON responses

❖User Flow – *LibSpace Backend*

The user flow describes how a user interacts with the system APIs step-by-step.

1️⃣User Registration

- User sends a **POST request** to `/api/auth/register`
- System validates input fields (name, email, password)
- Password is hashed using bcrypt
- User data is stored in the database
- Success response is returned

2️⃣User Login

- User sends a **POST request** to `/api/auth/login`
- System verifies email and password
- If valid, a **JWT token** is generated
- Token is sent back in the response
- User is now authenticated

3️⃣Access Protected Routes

User sends requests with:

```
Authorization: Bearer <token>
```

- Middleware verifies JWT
- If valid → request proceeds
- If invalid → access denied

4️⃣ Book Operations (After Login)

Authenticated user can:

- ➕ Add Book → POST `/api/book/add`
- 📖 Get Books → GET `/api/book`
- ✏️ Update Book → PUT `/api/book/:id`
- ❌ Delete Book → DELETE `/api/book/:id`

Each book is linked to the logged-in user.

5️⃣ Logout

- User sends request to `/api/auth/logout`
- Token is stored in blacklist
- User session becomes invalid

6️⃣ Delete User (Optional)

- User sends DELETE request with userId
- Account is removed from database

Project setup:

1. Install Dependencies

Install all required npm packages:

```
npm install
```

2. Configure Environment Variables

Create a `.env` file in the root directory and add the following:

```
PORT=5000

MONGO_URI=your_database_connection_string

JWT_SECRET=your_secret_key
```

- Replace
- `your_database_connection_string` with your MongoDB/MySQL connection string.
- Replace `your_secret_key` with any secure random string.

3. Start the Database

- Make sure MongoDB/MySQL is installed and running on your system.
- Verify the connection string matches your database configuration.

4. Run the Server

Start the application using:

```
npm start
```

Or (if using nodemon):

```
npm run dev
```

5. Access the Application

Open your browser and navigate to:

http://localhost:5000

The LibSpace application should now be running successfully.

❖ Backend Development

The backend of LibSpace is developed using **Node.js and Express.js**, following RESTful architecture principles. It handles authentication, business logic, database operations, and API responses. The backend ensures secure data handling, structured routing, and efficient communication with MongoDB.

1️⃣ API Development & Routing

2️⃣ Database Integration &

CONFIGURE MongoDB:

MongoDB is configured using **Mongoose**, an ODM (Object Data Modeling) library for Node.js.

Steps:

- Install mongoose:

```
npm install mongoose
```

- Create a database connection file inside `Config/db.js`
- Connect using MongoDB URI

Environment variable (.env):

```
MONGO_URI=mongodb://127.0.0.1:27017/libspace
```

CREATE DATABASE CONNECTION

File: Config/db.js

```
const mongoose = require("mongoose");

const connection = mongoose.connect(process.env.MONGO_URI);

module.exports = {

  connection,

};
```

In index.js:

```
const { connectionDB } = require("./Config/db");



app.listen(5000, async () => {

  try {

    await connectionDB;

    console.log("Connected to Database");

  } catch (err) {

    console.log("Database connection failed");

  }

});
```

This ensures the server starts only after a successful database connection.

CREATE SCHEMA AND MODELS:

Schemas define the structure of documents inside MongoDB collections.

User Schema

File: models/user.model.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    email: { type: String, required: true },
    password: { type: String, required: true },
  },
  { versionKey: false, timestamps: true }
);

const UserModel = mongoose.model("User", userSchema);

module.exports = UserModel;
```

Book Schema:

File: `models/book.model.js`

```
const mongoose = require("mongoose");

const bookSchema = new mongoose.Schema({
  url: { type: String, required: true },
  user: {
```

```
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  title: { type: String, required: true },
  author: { type: String, required: true },
});

const BookModel = mongoose.model("Book", bookSchema);

module.exports = BookModel;
```

Project Execution:

Project execution steps:

1.Install Dependencies

Open terminal inside the project folder and run:

```
npm install
```

This installs all required packages from package.json.

2. Start the Server

Run the following command:

npm start

OR (if using nodemon):

npm run dev

3. Database Connection

When the server starts:

- connectDb() connects to MongoDB.
- If successful, the console displays:

Connected to Database

Our server is working at PORT :5000

4.**Test API Endpoints**

Use Postman or any API testing tool to test the endpoints:

🔹 User Routes

- POST → /api/auth/register
- POST → /api/auth/login
- DELETE → /api/auth/delete

🔹 Book Routes

- POST → /api/book/add
- GET → /api/book/
- PUT → /api/book/:id

- DELETE → /api/book/:id

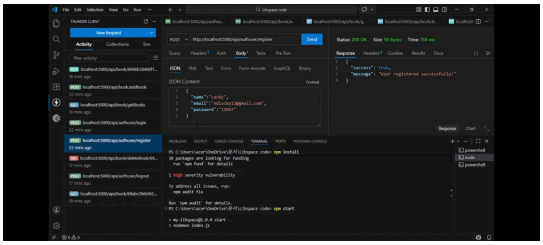
Base URL:

http://localhost:5000

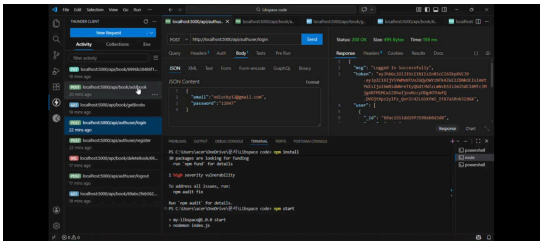
Output Screenshots:

Output on terminal once server starts:

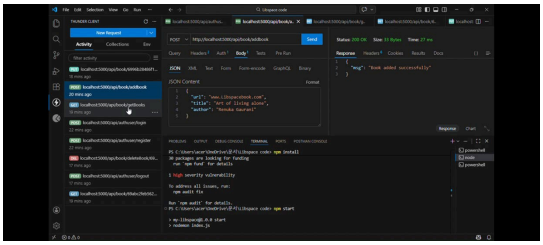
User registration output:



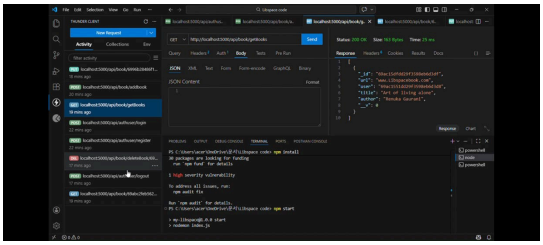
User Login output:



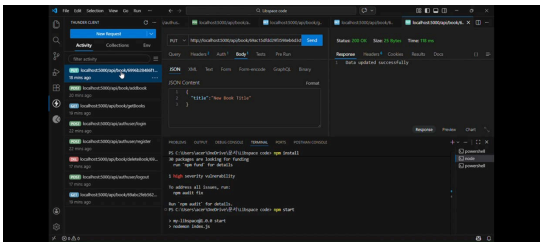
Adding book:



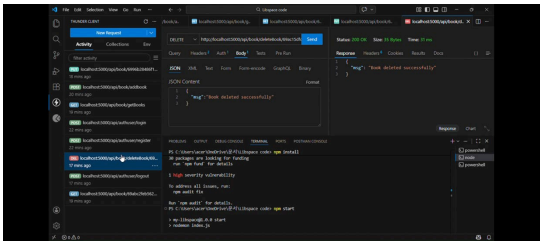
Get All books:



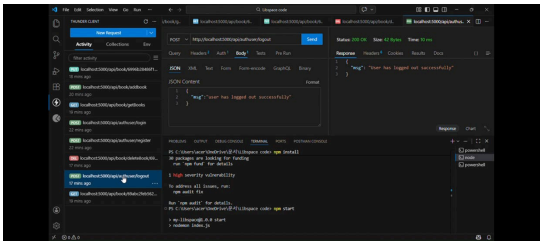
Update book:



Delete book:



User Logout:



Demo video:https://drive.google.com/file/d/1FKrsFOvESMMYjl4_tG2n3V8ucqXjY2nx/view?usp=drivesdk